

# Automatisiertes Modell-Hochregallager

**T3\_3200**

Elektrotechnik

Automation

Duale Hochschule Baden-Württemberg Ravensburg, Campus Friedrichshafen

von

Maximilian Zorko und Willi Schaal

Abgabedatum:

Bearbeitungszeitraum:

Matrikelnummer: 3960407/5732737

Kurs: TEA 23

Betreuerin / Betreuer: Prof. Dr. Ing. Thorsten Kever

Copyrightvermerk:

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

# Inhaltsverzeichnis

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                         | <b>1</b>  |
| 1.1      | Motivation und Problemstellung . . . . .                  | 1         |
| 1.2      | Bedeutung der Lagerlogistik . . . . .                     | 1         |
| 1.3      | Versuchsaufbau und bisherige Arbeiten . . . . .           | 1         |
| 1.4      | Zielsetzung der Arbeit . . . . .                          | 2         |
| <b>2</b> | <b>Grundlagen</b>                                         | <b>2</b>  |
| 2.1      | Problemstellung . . . . .                                 | 2         |
| 2.2      | Lagerprozesse . . . . .                                   | 3         |
| 2.3      | Auftragsbasierte Prozesse . . . . .                       | 3         |
| 2.3.1    | Grundprinzip der auftragsbasierten Lagerung . . . . .     | 3         |
| 2.3.2    | Ablauf der auftragsbasierten Lagerung . . . . .           | 4         |
| 2.4      | Identifikationssysteme (Scanner) . . . . .                | 6         |
| 2.4.1    | Allgemeine Einordnung . . . . .                           | 6         |
| 2.4.2    | Scanner als Identifikationssystem . . . . .               | 6         |
| 2.4.3    | Funktionsweise eines Scanners . . . . .                   | 7         |
| 2.4.4    | Datenübertragung / Schnittstelle . . . . .                | 8         |
| 2.4.5    | Vorteile beim Einsatz eines Scanners . . . . .            | 10        |
| 2.5      | Programmanwendung zur Datenverarbeitung . . . . .         | 10        |
| 2.6      | Speicherprogrammierbare Steuerungen (SPS) . . . . .       | 10        |
| 2.6.1    | Aufgaben . . . . .                                        | 11        |
| 2.6.2    | Aufbau . . . . .                                          | 11        |
| 2.6.3    | Eigenschaften . . . . .                                   | 12        |
| 2.6.4    | Bezug zur auftragsbasierten Steuerung . . . . .           | 12        |
| 2.7      | Kommunikation zwischen Python und SPS . . . . .           | 12        |
| 2.7.1    | Allgemeine Einordnung . . . . .                           | 12        |
| 2.7.2    | Ziel der Kommunikation . . . . .                          | 13        |
| 2.7.3    | Kommunikationsarten . . . . .                             | 14        |
| 2.7.4    | Kommunikationsprotokoll Snap7 . . . . .                   | 14        |
| <b>3</b> | <b>Konzept der auftragsbasierten Auslagerung</b>          | <b>15</b> |
| 3.1      | Systemaufbau . . . . .                                    | 15        |
| 3.2      | Gesamtablauf der auftragsbasierten Auslagerung . . . . .  | 18        |
| 3.2.1    | Anbindung der Materialdaten . . . . .                     | 18        |
| 3.2.2    | Aufbau der Programmanwendung . . . . .                    | 19        |
| 3.2.3    | Verarbeitung der Materialdaten . . . . .                  | 19        |
| 3.2.4    | Auslagerprozess innerhalb der Programmanwendung . . . . . | 21        |
| 3.2.5    | Einlesen von Daten aus der SPS . . . . .                  | 22        |

|       |                                                               |    |
|-------|---------------------------------------------------------------|----|
| 3.2.6 | Übertragung von Daten an die SPS . . . . .                    | 22 |
| 3.2.7 | Erfassung von Scannerdaten . . . . .                          | 22 |
| 3.3   | Anpassungen in der SPS . . . . .                              | 23 |
| 3.3.1 | Datenbausteine im Kontext der auftragsbasierten Auslagerung . | 23 |
| 3.3.2 | Anpassung der Prozessschrittkette "Auslagern" . . . . .       | 25 |

# 1 Einleitung

## 1.1 Motivation und Problemstellung

In automatisierten Industrieprozessen ist eine leistungsfähige und strukturierte Infrastruktur erforderlich, um einen reibungslosen und effizienten Ablauf zu gewährleisten. Solche Systeme sind hochgradig diversifiziert, da unterschiedliche Teilprozesse und Aufgaben gemeinsam zur Realisierung eines übergeordneten Produktions- oder Logistikablaufs beitragen.

Fehlen geeignete Strukturen oder sind diese unzureichend ausgelegt, kann dies zu Produktionsfehlern, Lieferverzögerungen oder ungeplanten Störungen innerhalb des Prozesszyklus führen. Eine zentrale Rolle nimmt in diesem Zusammenhang die **Logistik** ein. Logistische Prozesse tragen maßgeblich zur Stabilität, Kontinuität und Flexibilität industrieller Abläufe bei.

## 1.2 Bedeutung der Lagerlogistik

Innerhalb der Fertigung müssen Materialien, Bauteile und Produkte bedarfsgerecht bereitgestellt werden. Dies erfolgt über strukturierte logistische Prozesse, die eine zeit- und ortsgerechte Bereitstellung sicherstellen. Dabei spielt insbesondere die Lagerlogistik eine entscheidende Rolle.

In diesem Bereich werden Materialien organisiert gelagert, verwaltet und für nachgelagerte Prozesse verfügbar gemacht. Häufig erfolgt dies in Form von Hochregallagern, die eine platzsparende und automatisierte Lagerhaltung ermöglichen.

## 1.3 Versuchsaufbau und bisherige Arbeiten

Zur praxisnahen Abbildung solcher Prozesse steht an der **DHBW Ravensburg – Campus Friedrichshafen** ein didaktisches Modell eines Hochregallagers zur Verfügung. Dieses System ermöglicht es Studierenden, praktische Erfahrungen im Bereich der automatisierten Logistik zu sammeln.

In der vorangegangenen Arbeit im Modul **T3\_3100** wurden bereits Anpassungen und Erweiterungen an diesem System vorgenommen. Der Fokus lag dabei insbesondere auf sicherheitsrelevanten Aspekten sowie auf dem Prozess des **Umlagens**.

### 1.4 Zielsetzung der Arbeit

Die vorliegende Arbeit baut auf diesen Grundlagen auf und erweitert das bestehende System um eine realitätsnahe Funktionalität:

Die **auftragsbasierte Steuerung des Auslagerprozesses**.

In modernen industriellen Lagerverwaltungssystemen stellt die auftragsbasierte Prozesssteuerung einen wesentlichen Bestandteil dar, um Effizienz, Nachvollziehbarkeit und Flexibilität zu gewährleisten.

Ziel dieser Arbeit ist es, ein entsprechendes Steuerungskonzept zu entwickeln und auf das vorhandene Modell zu übertragen. Dabei wird insbesondere die Umsetzung einer auftragsgesteuerten Auslagerung unter Einsatz von **Python** betrachtet. Die Integration der Softwarelösung ermöglicht eine flexible Erweiterung der bestehenden Steuerungsarchitektur sowie eine verbesserte Visualisierung und Verarbeitung von Prozessdaten.

## 2 Grundlagen

### 2.1 Problemstellung

In der vorangegangenen Arbeit, **T3\_3100**, wurden bereits grundlegende Funktionen des Hochregallagers implementiert und erweitert. Dazu zählen insbesondere die Prozesse:

- Einlagern
- Auslagern
- Umlagern

Die Ein- und Auslagerprozesse wurden dabei vollständig umgesetzt, während die Funktion **Umlagern** im Rahmen der genannten Arbeit neu eingeführt wurde.

Im Fokus der vorliegenden Arbeit steht nun die Erweiterung des bestehenden Auslagerprozesses hin zu einer **auftragsbasierten Auslagerung**. Ziel ist es, eine realitätsnahe Abbildung industrieller Logistikprozesse zu ermöglichen, bei der Auslagerungen nicht mehr manuell, sondern basierend auf konkreten Aufträgen erfolgen.

Zur Umsetzung dieses Ansatzes wird ein **Scanner** eingesetzt, welcher das Einlesen von Bauteil- oder Artikelcodes ermöglicht. Diese Codes dienen als Grundlage für die Zuordnung und Identifikation von Lagergütern innerhalb des Systems.

Ergänzend dazu wird eine **externe Datenbankanbindung** realisiert, welche eine Synchronisation zwischen den erfassten Scandaten und den zugehörigen Auftragsinformationen erlaubt. Dadurch kann eine automatisierte und auftragsbezogene Auslagerstrategie umgesetzt werden.

Ein wesentlicher Bestandteil dieser Erweiterung ist der Einsatz der Programmiersprache **Python**. Im Gegensatz zur reinen SPS-Programmierung ermöglicht Python die Verarbeitung komplexerer Datenstrukturen sowie eine flexible Anbindung externer Systeme, wie beispielsweise Datenbanken oder Visualisierungstools.

Durch die Auslagerung von komplexeren Logik- und Datenverarbeitungsprozessen in eine Hochsprache kann die Programmstruktur auf der SPS bewusst vereinfacht gehalten werden. Dies führt zu einer besseren Überschaubarkeit, Wartbarkeit und Erweiterbarkeit des Gesamtsystems.

Im folgenden Abschnitt wird der Stand der Technik betrachtet. Dabei werden die wesentlichen Grundlagen und Technologien vorgestellt, die für die Umsetzung der auftragsbasierten Auslagerung erforderlich sind. Die grundlegenden Lagerprozesse selbst werden an dieser Stelle nicht erneut behandelt, da diese bereits in der Dokumentation zur **T3\_3100** ausführlich beschrieben sind.

## 2.2 Lagerprozesse

## 2.3 Auftragsbasierte Prozesse

### 2.3.1 Grundprinzip der auftragsbasierten Lagerung

Die auftragsbasierte Lagerung ermöglicht es dem Benutzer, den Lagerprozess nicht mehr durch direkte Eingaben, sondern über definierte Aufträge zu steuern.

Im Falle der manuellen Lagerung gibt der Benutzer seinen gewünschten Lagerplatz vor. Alternativ besteht die Möglichkeit, ein bestimmtes Bauteil auszuwählen, woraufhin der entsprechende Prozess initiiert wird.

Bei der auftragsbasierten Lagerung wird der Lagerprozess hingegen über ein **Eingabegerät** gesteuert. Dabei erfolgt die Eingabe nicht mehr positionsorientiert, sondern auftragsorientiert.

Ein Auftrag enthält die notwendigen Informationen zur Identifikation eines Lagergutes, beispielsweise eine Artikel- oder Bauteilnummer. Auf Grundlage dieser Informationen übernimmt das System die weitere Verarbeitung und trifft eigenständig Entscheidungen hinsichtlich der Lagerbewegung.

In vielen Fällen werden diese Informationen über eine Datenbank bereitgestellt, wodurch eine flexible und modulare Anpassung der Auftragsstruktur jederzeit möglich ist.

Die logische Verarbeitung der Aufträge erfolgt dabei softwareseitig, während die physische Umsetzung weiterhin durch die speicherprogrammierbare Steuerung (SPS) realisiert wird. Dadurch entsteht eine klare Trennung zwischen Entscheidungslogik und Anlagensteuerung.

Der Ablauf eines solchen Prozesses beginnt mit der Erfassung eines Auftrags, beispielsweise durch das Einlesen eines Codes mittels Scanner. Anschließend wird dieser Auftrag verarbeitet und die entsprechenden Lageroperationen werden automatisch ausgelöst.

Ziel dieser Vorgehensweise ist es, manuelle Eingriffe zu reduzieren, die Prozesseffizienz zu steigern sowie eine höhere Nachvollziehbarkeit und Fehlerreduktion innerhalb der Lagerlogistik zu gewährleisten.

### **2.3.2 Ablauf der auftragsbasierten Lagerung**

Der Ablauf der auftragsbasierten Lagerung basiert auf dem Zusammenspiel mehrerer Systemkomponenten, welche unterschiedlichen Ebenen zugeordnet sind[7]. Diese Ebenen umfassen sowohl die Eingabe- und Verarbeitungsebene als auch die Steuerungs- und Ausführungsebene der Anlage. Ziel dieser Struktur ist es, die einzelnen Prozessschritte klar voneinander zu trennen und die Datenflüsse innerhalb des Systems transparent darzustellen.

Im Gegensatz zu klassischen Lagerprozessen erfolgt die Steuerung hierbei nicht direkt durch den Bediener, sondern durch eine softwaregestützte Verarbeitung von Auftragsdaten. Diese Daten werden zunächst erfasst, anschließend verarbeitet und schließlich an die Steuerungsebene übergeben.

In Abbildung 3.3 ist der grundlegende Aufbau des Systems dargestellt. Der Signalfluss beginnt bei der Eingabe, beispielsweise durch einen Scanner, und verläuft über die Verarbeitungsebene, welche durch eine Programmanwendung (z.B. Python) realisiert wird. Innerhalb dieser Ebene erfolgt auch die Synchronisation mit einer Datenbank sowie die Visualisierung über ein HMI. Anschließend werden die verarbeiteten Daten an die SPS übertragen, welche den physikalischen Teil des Prozesses übernimmt.

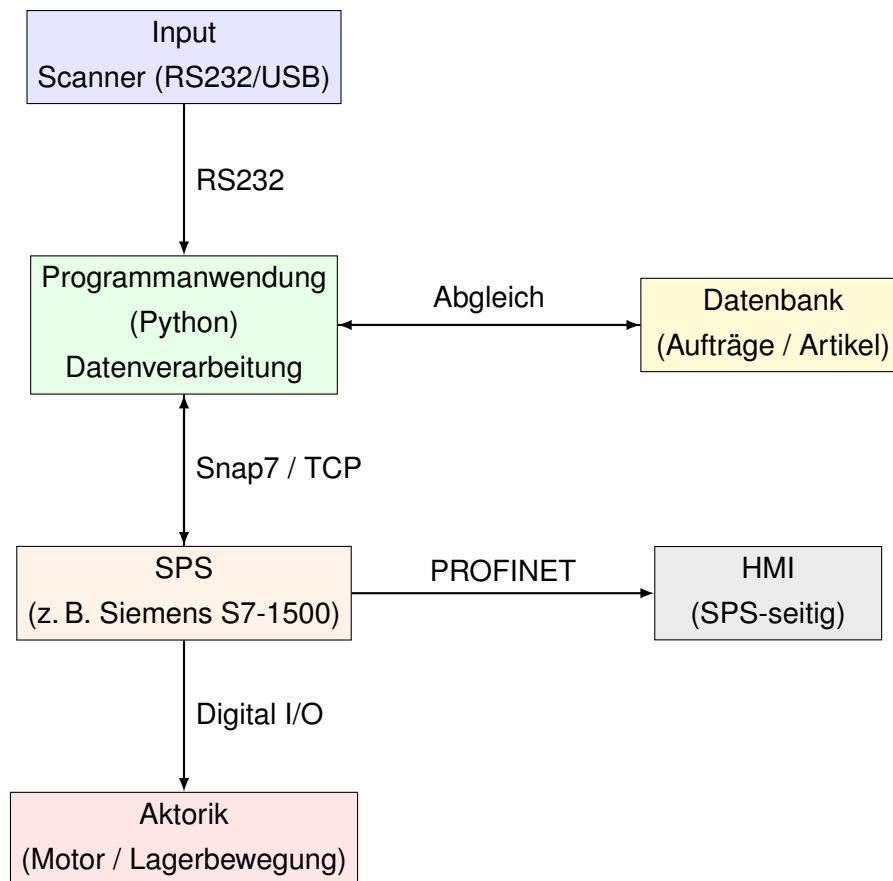


Abbildung 2.1: Systemaufbau der auftragsbasierten Lagersteuerung

Wie in Abbildung 3.3 ersichtlich, erfolgt der Ablauf weitgehend seriell. Das bedeutet, dass die einzelnen Prozessschritte nacheinander durchlaufen werden und die Daten schrittweise von der Eingabe bis zur SPS weitergegeben werden. Die SPS stellt dabei die zentrale Schnittstelle zwischen der softwarebasierten Verarbeitungsebene und der physikalischen Anlagenebene dar.

Im ersten Schritt erfolgt die Eingabe eines Bauteilcodes, beispielsweise durch das Scannen eines Barcodes. Diese Eingabe wird von der Programmanwendung erfasst und weiterverarbeitet.

In der Verarbeitungsebene werden die eingelesenen Daten bereinigt und interpretiert. Auf Basis des Codes erfolgt ein Abgleich mit den in der Datenbank hinterlegten Informationen. Dadurch kann bestimmt werden, welches Bauteil ausgelagert werden soll und welche Lagerposition zugeordnet ist.

Nach erfolgreicher Verarbeitung werden die benötigten Steuerinformationen über eine geeignete Kommunikationsschnittstelle an die SPS übertragen. Hierbei kommen je nach System beispielsweise Protokolle wie Snap7 oder ADS zum Einsatz.



Die SPS übernimmt anschließend die Ansteuerung der Aktorik. Diese führt die physikalische Bewegung innerhalb des Lagersystems aus, beispielsweise das Verfahren eines Motors oder das Bewegen eines Zylinders zur Auslagerung des gewünschten Bauteils.

Die folgenden Abschnitte orientieren sich am in Abbildung 3.3 dargestellten Systemaufbau und beschreiben die einzelnen Komponenten sowie deren Funktion innerhalb des Gesamtsystems.

## 2.4 Identifikationssysteme (Scanner)

### 2.4.1 Allgemeine Einordnung

Die eindeutige Identifikation von Bauteilen ist ein grundlegender Bestandteil logistischer Prozesse. Diese Identifikation kann über unterschiedlichste Verfahren erfolgen. Die gängigste Prüfung erfolgt durch **Scanner**.

Automatisierte Systeme benötigen eine klare Zuordnung von Objekten um richtig arbeiten zu können. In der Logistik ist dies von besonders hoher Bedeutung.

Ohne eine solche Identifikation ist keine auftragsbasierte Steuerung möglich und es kann zu erheblichen Komplikationen kommen.

### 2.4.2 Scanner als Identifikationssystem

Die Aufgabe eines Einlesegerätes besteht darin, Daten schnell und zuverlässig zu erfassen. Zum Einlesen beziehungsweise zur Eingabe von Daten stehen verschiedene Möglichkeiten zur Verfügung:

- Eingabe über Tastatur
- Einlesen über ein Scan-Gerät
- Erkennung des Bauteils über eine Kamera

Im Logistikbereich müssen Daten in der Regel schnell und fehlerfrei übertragen werden. Für diese Anwendung eignen sich Scanner besonders gut. Diese sind einfach in der Handhabung und ermöglichen eine schnelle Erfassung von Daten. Im Gegensatz zur Eingabe über die Tastatur ist keine manuelle Eingabe erforderlich, wodurch Fehler reduziert werden können.

Zur Identifikation werden Codierungen benötigt. In der Regel werden hierfür Barcodes oder QR-Codes verwendet.

Im Folgenden ist eine Übersicht über die wichtigsten Merkmale und Unterschiede von **QR-Codes** und **Barcodes** dargestellt:

| <b>Merkmal</b>     | <b>Barcode</b>          | <b>QR-Code</b>                        |
|--------------------|-------------------------|---------------------------------------|
| Struktur           | Eindimensional (1D)     | Zweidimensional (2D)                  |
| Datenspeicher      | Geringe Datenmenge      | Hohe Datenmenge                       |
| Leserichtung       | Nur horizontal lesbar   | Aus allen Richtungen lesbar           |
| Fehlerkorrektur    | Kaum vorhanden          | Integrierte Fehlerkorrektur           |
| Verwendung         | Artikeletiketten, Lager | Industrie, Tracking, Datenübertragung |
| Informationsgehalt | Meist nur ID / Nummer   | Kann komplexe Daten enthalten         |

Tabelle 1: Vergleich von Barcode- und QR-Code-Systemen; Quelle: [10]

In Tabelle 1 werden die wesentlichen Unterschiede zwischen Barcode- und QR-Code-Systemen dargestellt. Während Barcodes aufgrund ihrer einfachen Struktur hauptsächlich zur Identifikation von Artikeln verwendet werden, bieten QR-Codes aufgrund ihres zweidimensionalen Aufbaus die Möglichkeit, deutlich mehr Informationen zu speichern.

Für industrielle Anwendungen, wie beispielsweise in automatisierten Lagersystemen, werden in vielen Fällen weiterhin Barcodes eingesetzt, da diese für die eindeutige Identifikation von Bauteilen ausreichend sind und sich durch eine einfache Implementierung auszeichnen. In komplexeren Anwendungen können jedoch auch QR-Codes verwendet werden, insbesondere wenn

### 2.4.3 Funktionsweise eines Scanners

Der Einlesevorgang bei einem Scanner erfolgt über ein optisches Verfahren, welches je nach Bauart auf Licht-, Laser- oder Kameratechnologie basiert. Beim Lichtverfahren wird das Objekt gleichmäßig ausgeleuchtet, während beim Laserverfahren ein gebündelter Lichtstrahl über den Code geführt wird. Kamerabasierte Systeme arbeiten hingegen mit Bildsensoren und erfassen den Code flächenhaft.

Allen Verfahren gemeinsam ist die Auswertung von Hell-Dunkel-Unterschieden, welche die codierten Informationen repräsentieren.

Der Ablauf des Einlesevorgangs lässt sich wie folgt beschreiben:

- 1. Beleuchtung des Codes durch Licht- oder Laserquelle
- 2. Erfassung der reflektierten Lichtanteile durch einen Sensor
- 3. Unterscheidung zwischen hellen und dunklen Bereichen des Codes
- 4. Umwandlung der optischen Signale in elektrische Signale
- 5. Verarbeitung und Interpretation als Zahlen- oder Zeichenfolge

Die erzeugte Zeichenfolge entspricht dabei der im Code enthaltenen Information, beispielsweise einer Artikelnummer oder Bauteilkennung. Diese kann anschließend von nachgelagerten Systemen weiterverarbeitet werden.



Abbildung 2.2: Kamerabasierter Scanner[1]

### 2.4.4 Datenübertragung / Schnittstelle

Die Datenübertragung erfolgt in der Regel seriell an ein übergeordnetes System. Die erfassten Daten werden unmittelbar nach dem Einlesevorgang an die Steuerung, beispielsweise einen PC oder eine SPS, weitergeleitet. Die Ausgabe erfolgt üblicherweise als Zeichenkette (String).

Je nach Datenformat sind Anpassungen innerhalb der Steuerung oder durch eine zwischengeschaltete Software erforderlich, um eine korrekte Weiterverarbeitung der Daten zu gewährleisten. Am Ende des Scanwertes befindet sich in der Regel ein Abschlusszeichen, beispielsweise "\n" oder ein "Enter"-Signal.

Die **Schnittstelle** kann je nach Scannerart unterschiedlich ausgeführt sein. Häufig werden RS232 sowie USB verwendet. Während RS232 eine klassische serielle Schnittstelle darstellt, kann USB durch die Emulation eines virtuellen COM-Ports ebenfalls wie eine serielle Schnittstelle verwendet werden.

Im Folgenden sind die wichtigsten Eigenschaften und Unterschiede beider Anschlussarten dargestellt:

| <b>Merkmal</b>  | <b>RS232</b>                | <b>USB</b>                              |
|-----------------|-----------------------------|-----------------------------------------|
| Übertragungsart | Klassisch seriell           | Paketbasiert (virtuell seriell möglich) |
| Schnittstelle   | Physischer COM-Port         | Virtueller COM-Port (häufig)            |
| Datenrate       | Niedrig bis mittel          | Hoch                                    |
| Verzögerung     | Sehr gering                 | Gering                                  |
| Implementierung | Einfach, robust             | Etwas komplexer                         |
| Verwendung      | Industrie / Automatisierung | PC-Systeme / moderne Geräte             |

Tabelle 2: Vergleich der Schnittstellen RS232[5] und USB[2]

Aus Tabelle 2 wird deutlich, dass beide Schnittstellen ihre Berechtigung im industriellen Umfeld besitzen. Während RS232 besonders durch seine einfache und robuste Struktur überzeugt[5], bietet USB eine höhere Datenübertragungsrate und wird häufig durch virtuelle COM-Ports in bestehende Systeme integriert[2].

Schlussendlich werden die erfassten Daten über eine Programmanwendung, beispielsweise in Python oder C, weiterverarbeitet. Dabei können projektabhängige Anpassungen vorgenommen werden, um die Daten entsprechend aufzubereiten. Die eigentliche Prozesslogik wird somit softwareseitig realisiert, wodurch automatisierte Abläufe, wie beispielsweise die Zuordnung von Bauteilen anhand von Scanwerten, angestoßen werden können.

### 2.4.5 Vorteile beim Einsatz eines Scanners

Der Einsatz von Scannern bietet insbesondere im industriellen und logistischen Umfeld zahlreiche Vorteile. Im Folgenden sind die wichtigsten Aspekte übersichtlich dargestellt:

| Vorteil                   | Beschreibung                                                                            |
|---------------------------|-----------------------------------------------------------------------------------------|
| Schnelle Datenerfassung   | Scanvorgänge erfolgen in sehr kurzer Zeit und ermöglichen hohe Prozessgeschwindigkeiten |
| Reduzierte Fehlerquote    | Keine manuelle Eingabe notwendig, dadurch geringere Eingabefehler                       |
| Einfache Integration      | Scanner lassen sich leicht in bestehende Systeme einbinden                              |
| Hohe Zuverlässigkeit      | Robuste Technik, geeignet für industrielle Anwendungen                                  |
| Automatisierungspotenzial | Ermöglicht die direkte Weiterverarbeitung von Daten in automatisierten Prozessen        |
| Eindeutige Identifikation | Barcodes und QR-Codes ermöglichen klare Zuordnung von Bauteilen                         |
| Flexibilität              | Einsetzbar in verschiedenen Anwendungen und Branchen                                    |

Tabelle 3: Vorteile des Einsatzes von Scannern in automatisierten Systemen[1]

Wie in Tabelle 3 dargestellt, tragen Scanner maßgeblich zur Effizienzsteigerung in automatisierten Prozessen bei. Insbesondere die schnelle und fehlerarme Datenerfassung stellt einen entscheidenden Vorteil dar. Im Zusammenspiel mit softwarebasierten Verarbeitungssystemen können somit vollständig automatisierte Abläufe realisiert werden.

## 2.5 Programmanwendung zur Datenverarbeitung

### 2.6 Speicherprogrammierbare Steuerungen (SPS)

Speicherprogrammierbare Steuerungen (SPS) werden in der Industrie zur Automatisierung von Prozessen eingesetzt und bilden die zentrale Steuereinheit vieler Anlagen. Sie dienen als Ersatz für aufwendig zu implementierende Relais- oder Schützschaltungen. Durch ihren Einsatz können schnellere und effizientere Abläufe innerhalb eines

Prozesses realisiert werden.

In der modernen Industrie sind kurze Zykluszeiten erforderlich, welche mit rein hardwarebasierten Schaltungen nur schwer erreicht werden können. Eine SPS arbeitet nach einem zyklischen Prinzip, wodurch sowohl einfache als auch komplexe Prozesse in kurzen Zeitabständen ausgeführt werden können.

Die zyklische Abarbeitung eines Programms bedeutet, dass ein Programmablauf von Beginn bis Ende sequenziell durchlaufen und anschließend wiederholt wird. Dieser Vorgang wird als **Zyklus** bezeichnet. Die benötigte Zykluszeit ist dabei stark von der jeweiligen Anwendung abhängig. In logistischen Prozessen beeinflusst sie beispielsweise die Effizienz von Ein- und Auslagerungen, während in der Fertigung oftmals deutlich kürzere Zykluszeiten erforderlich sind[8].

### 2.6.1 Aufgaben

Eine klassische SPS arbeitet nach dem **EVA-Prinzip** (Eingabe – Verarbeitung – Ausgabe)[9].

Zu Beginn werden Eingänge wie Sensoren, Schalter oder Trigger, beispielsweise von einem Scanner, eingelesen. Diese Eingangssignale werden anschließend innerhalb eines Programms verarbeitet. Im letzten Schritt erfolgt die gezielte Ansteuerung der Ausgänge, beispielsweise zur Steuerung von Aktoren.

### 2.6.2 Aufbau

Der Aufbau einer SPS ist in der Regel kompakt und modular gestaltet. Im Wesentlichen besteht eine SPS aus folgenden Komponenten[11]:

- CPU
- Ein- und Ausgangsmodule
- Kommunikationsschnittstellen

Die zentrale Verarbeitung erfolgt innerhalb der CPU. Diese verarbeitet die Eingangssignale und steuert auf Basis des Programms die Ausgänge an. Kommunikationsschnittstellen ermöglichen die Verbindung zu externen Systemen, beispielsweise zu einem **Human Machine Interface (HMI)** oder einem externen Rechner.

### 2.6.3 Eigenschaften

Speicherprogrammierbare Steuerungen weisen eine Reihe spezifischer Eigenschaften auf. Sie sind besonders robust und für den industriellen Einsatz ausgelegt. Eine wesentliche Eigenschaft ist die **Echtzeitfähigkeit**, welche die Einhaltung definierter Zykluszeiten sicherstellt[8].

Zusätzlich verfügen viele moderne SPS-Systeme über redundante Komponenten. Dies bedeutet, dass wichtige Systemteile mehrfach vorhanden sind, um die Funktionsfähigkeit der Anlage auch im Fehlerfall aufrechtzuerhalten[8].

### 2.6.4 Bezug zur auftragsbasierten Steuerung

Im Rahmen der auftragsbasierten Steuerung übernimmt die SPS die Ausführung des physikalischen Prozesses. Die übergeordneten Steuerinformationen werden dabei von einer externen Programmanwendung bereitgestellt.

Ziel ist es, die SPS-Programmierung möglichst einfach zu halten. Komplexe Datenverarbeitungsaufgaben, wie beispielsweise die Verarbeitung von Datenbankinhalten oder die Kommunikation mit externen Systemen, werden dabei auf einen externen Rechner ausgelagert.

In der industriellen Praxis werden jedoch häufig Kompromisse eingegangen. Teilweise werden auch komplexere Datenstrukturen direkt innerhalb der SPS verarbeitet. Dies kann jedoch die Wartbarkeit und Fehlersuche erschweren und sollte daher nur in begrenztem Umfang erfolgen.

Moderne SPS-Systeme bieten die Möglichkeit, mit externen Systemen zu kommunizieren. So können beispielsweise Programme in **Python** verwendet werden, um Daten aus der SPS auszulesen oder Variablen gezielt zu setzen. Dadurch entsteht eine flexible Systemarchitektur, bei der Software- und Steuerungsebene effektiv miteinander kombiniert werden[3].

## 2.7 Kommunikation zwischen Python und SPS

### 2.7.1 Allgemeine Einordnung

Zur Kommunikation zwischen einer Programmanwendung und einer speicherprogrammierbaren Steuerung werden spezielle Kommunikationsprotokolle eingesetzt, welche

den Datenaustausch zwischen beiden Systemen ermöglichen.

Moderne Automatisierungssysteme sind hierbei in mehrere Ebenen unterteilt:

- Steuerungsebene (SPS)
- IT-/Softwareebene (PC, Python)

Innerhalb der Steuerungsebene wird der Hauptprozess mithilfe von verknüpften Ein- und Ausgangssignalen gesteuert. Die IT- bzw. Softwareebene ermöglicht hingegen einen gezielten Austausch und die Verarbeitung von Informationen, beispielsweise die Verwaltung von Lagerdaten oder Aufträgen[4].

Der Datenaustausch umfasst dabei verschiedene Informationsarten:

- Steuerbefehle
- Statusinformationen
- Prozessdaten

Durch die Übergabe dieser Informationen kann ein dynamisches und automatisiertes Handling von Prozessen realisiert werden[4].

### 2.7.2 Ziel der Kommunikation

Die Übertragung von Daten von einer Programmanwendung, beispielsweise in **Python**, an die SPS ermöglicht eine weitergehende Automatisierung von Prozessen. Dabei entfällt die manuelle Eingabe von Werten, wodurch Fehler reduziert und Abläufe beschleunigt werden können.

Die SPS übernimmt weiterhin die kontinuierliche Steuerung der Anlage, während komplexere Datenverarbeitungsprozesse extern abgewickelt werden. Zusätzlich besteht die Möglichkeit, Status- und Fehlermeldungen aus der SPS auszulesen und weiterzuverarbeiten.

Ein beispielhafter Ablauf gestaltet sich wie folgt:

Scan → Python → SPS → Auslagerung



### 2.7.3 Kommunikationsarten

Für den erfolgreichen Austausch von Informationen ist eine geeignete Kommunikationsart erforderlich. Grundsätzlich lassen sich zwei wesentliche Arten unterscheiden:

- Serielle Kommunikation (z.B. RS232)
- Netzwerkbasierte Kommunikation (z.B. TCP/IP)

Serielle Schnittstellen werden häufig für die direkte Anbindung von Geräten wie Scannern verwendet. Für die Kommunikation zwischen einer Programmanwendung und der SPS wird in der Regel eine netzwerkbasierte Kommunikation über Ethernet eingesetzt. Diese ermöglicht einen schnellen und flexiblen Datenaustausch zwischen mehreren Systemteilnehmern[4].

### 2.7.4 Kommunikationsprotokoll Snap7

Für den Datenaustausch zwischen einer Siemens-SPS und externen Anwendungen stehen verschiedene Kommunikationsmöglichkeiten zur Verfügung. Eine verbreitete Lösung ist die Open-Source-Bibliothek **Snap7**.

Snap7 basiert auf dem S7-Kommunikationsprotokoll und ermöglicht es, von externen Anwendungen – beispielsweise in Python – direkt auf Speicherbereiche der SPS zuzugreifen[6].

Zu den wichtigsten Funktionen von Snap7 zählen:

- Lesen von Daten aus der SPS
- Schreiben von Daten in die SPS
- Zugriff auf Merker, Ein- und Ausgänge sowie Datenbausteine

Durch die Nutzung von Snap7 kann eine Programmanwendung gezielt Variablen innerhalb der SPS setzen oder auslesen. Dies ermöglicht beispielsweise das Starten eines Prozesses auf Basis eines eingelesenen Scanwertes[6].

Im Vergleich dazu verwendet der Hersteller Beckhoff das Kommunikationsprotokoll **ADS**, welches eine ähnliche Funktionalität bietet und ebenfalls den Zugriff auf Steuerungsvariablen erlaubt.

Die Verwendung solcher Protokolle ermöglicht eine klare Trennung zwischen Steuerungslogik und Datenverarbeitung. Während die SPS für die zeitkritische Anlagensteuerung verantwortlich ist, übernimmt die externe Software komplexe Datenverarbeitungsaufgaben[6].

Durch diese Struktur entsteht eine flexible und erweiterbare Systemarchitektur, welche besonders für auftragsbasierte Prozesse in der Logistik geeignet ist.

Die im vorliegenden Abschnitt beschriebenen Grundlagen bilden die Basis für die in den folgenden Kapitel dargestellte Konzeptentwicklung- und Umsetzung des Systems. Dabei wird insbesondere auf die konkrete Integration der einzelnen Komponenten eingegangen.

## 3 Konzept der auftragsbasierten Auslagerung

In diesem Kapitel wird das entwickelte Konzept zur auftragsbasierten Auslagerung dargestellt. Ziel ist es, den Aufbau des Systems sowie den Ablauf der Datenverarbeitung und die Interaktion der einzelnen Komponenten innerhalb des Gesamtsystems zu erläutern.

### 3.1 Systemaufbau

Zu Beginn der Konzeptentwicklung ist eine Betrachtung des Systemaufbaus von großer Bedeutung.

Im Rahmen der Entwicklung der auftragsbasierten Auslagerung wurden zunächst verschiedene mögliche Systemarchitekturen analysiert und miteinander verglichen.

Eine erste Möglichkeit bestand darin, die vollständige auftragsbasierte Steuerung direkt in der **speicherprogrammierbaren Steuerung (SPS)** umzusetzen. Dies hätte bedeutet, dass sowohl die Ansteuerung des Scanners als auch die gesamte Auftragslogik innerhalb der **SPS** realisiert wird.

Für diese Variante ergeben sich folgende Vor- und Nachteile:

#### Vorteile

- Zentrale Steuerung aller Funktionen (Einlagern, Auslagern, Umlagern) innerhalb der **SPS**
- Nur eine Softwarekomponente notwendig

#### Nachteile

- Geringe Übersichtlichkeit bei wachsendem Funktionsumfang
- Hoher Programmieraufwand innerhalb der **SPS**
- Komplexe Umsetzung bei der Verarbeitung strukturierter Daten (z. B. Listen oder Datenbanken)

Bei näherer Betrachtung zeigt sich, dass diese Variante insbesondere hinsichtlich **Wartbarkeit** und **Erweiterbarkeit** erhebliche Nachteile aufweist. Die Komplexität der Programmierung innerhalb der **SPS** würde stark ansteigen, wodurch Anpassungen und Fehlersuche erschwert werden.

Aus diesem Grund wurde eine zweite Systemarchitektur betrachtet. Bei dieser wird die **Auftragsverwaltung** vom eigentlichen **Steuerungsprozess** getrennt. Ein externer Rechner übernimmt die auftragsbasierte Verarbeitung der Daten, während die **SPS** weiterhin die physikalische Ausführung der Prozesse steuert.

Auch für diese Variante ergeben sich charakteristische Vor- und Nachteile:

#### Vorteile

- Hohe **Wartbarkeit** durch klare Trennung der Systemkomponenten
- Verbesserte **Übersichtlichkeit** durch Aufteilung in Funktionsbereiche
- Effiziente Datenverarbeitung durch Nutzung geeigneter Programmiersprachen (z. B. **Python**)
- Nutzung vorhandener Kommunikationsschnittstellen (z. B. RS232, TCP/IP)

#### Nachteile

- Höherer Integrationsaufwand durch Kommunikation zwischen mehreren Systemen
- Abhängigkeit von externer Software

Aufgrund der deutlich besseren **Wartbarkeit**, **Flexibilität** und **Erweiterbarkeit** wurde die zweite Variante für die Umsetzung des Systems gewählt. Insbesondere die Trennung von **Datenverarbeitung** und **Steuerungslogik** ermöglicht es, komplexe Aufgaben außerhalb der **SPS** zu realisieren und die Steuerung bewusst einfach zu halten.

Diese Architektur entspricht zudem dem Stand moderner **Automatisierungssysteme**, bei denen **IT- und Steuerungsebene** zunehmend miteinander vernetzt werden.

Im Folgenden ist der resultierende Systemaufbau dargestellt:

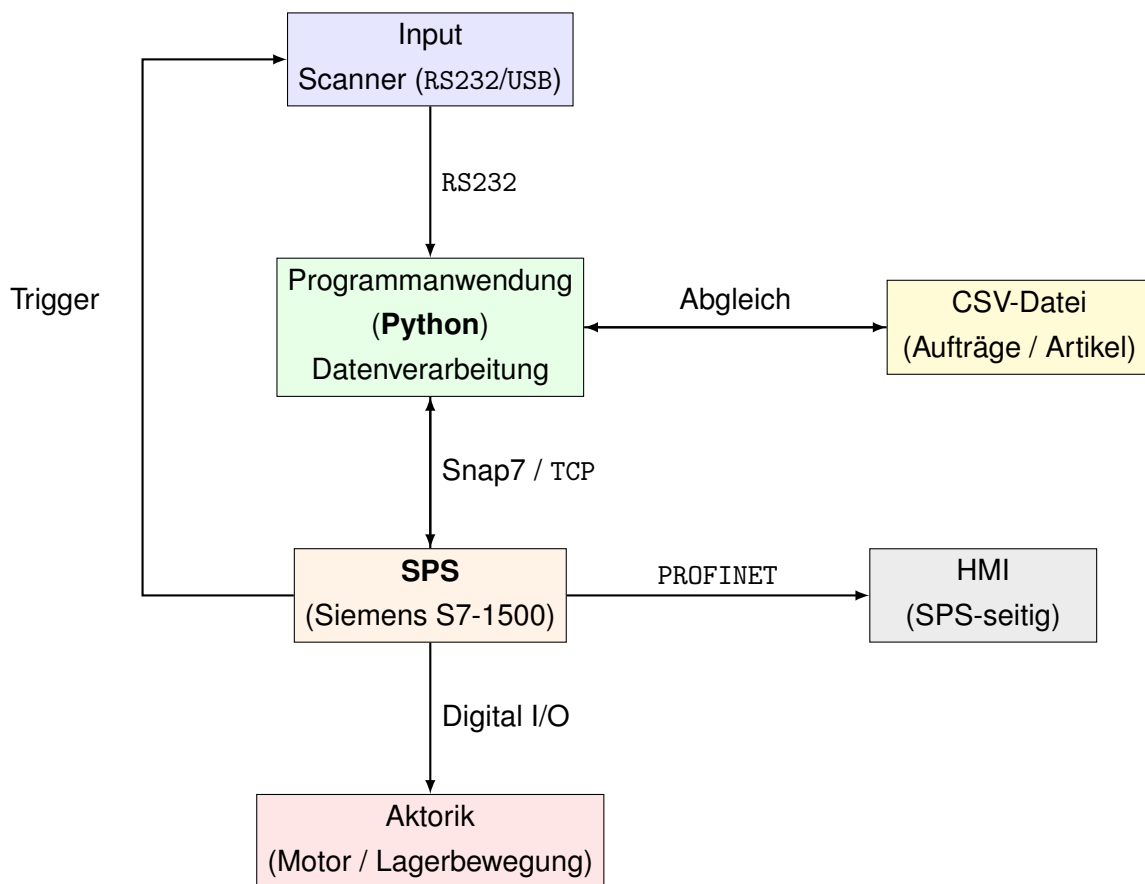


Abbildung 3.3: Systemaufbau der auftragsbasierten Lagersteuerung

Der im vorherigen Abschnitt dargestellte Systemaufbau beschreibt die strukturelle Anordnung der einzelnen Komponenten sowie deren Verknüpfung innerhalb des Gesamtsystems. Aufbauend auf dieser Architektur kann nun der konkrete Ablauf der auftragsbasierten Auslagerung betrachtet werden.

Im folgenden Abschnitt wird daher erläutert, wie die einzelnen Komponenten im Zusammenspiel agieren und welche Schritte vom Einlesen eines Bauteilcodes bis zur Ausführung der Auslagerung durchlaufen werden. Dabei wird insbesondere der Datenfluss innerhalb des Systems betrachtet.

## 3.2 Gesamtablauf der auftragsbasierten Auslagerung

Zu Beginn der Implementierung der Auftragssteuerung wurde das vorhandene System betrachtet.

Der Auslagerprozess wurde bereits in der Dokumentation der vorherigen Studienarbeit beschrieben und wird deshalb an dieser Stelle nicht erneut im Detail erläutert.

### 3.2.1 Anbindung der Materialdaten

Im bestehenden Konzept sind die verschiedenen Materialien innerhalb der SPS in Form einer Textliste hinterlegt. Diese Art der Datenhaltung ist jedoch nur eingeschränkt geeignet, da sie eine Erweiterung sowie eine flexible Verarbeitung der Daten erschwert.

Aus diesem Grund wurde eine externe Datenstruktur eingeführt, welche auf einem separaten Rechner gespeichert wird. Diese kann beispielsweise in Form einer **CSV-Datei** realisiert werden und enthält die relevanten Materialinformationen.

| A       | B            | C            | D            |
|---------|--------------|--------------|--------------|
| QR Code | Materialname | Lagerplatz x | Lagerplatz y |
| 5964135 | Eiche        | 2.0          | 1.0          |
| 1041573 | Kiefer       | 3.0          | 1.0          |
| 1042019 | Buche        | 3.0          | 2.0          |
|         |              |              |              |
|         |              |              |              |

Abbildung 3.4: Beispielhafte Struktur der Materialdaten

In Abbildung 3.4 ist ein exemplarischer Aufbau der verwendeten Materialdaten dargestellt. Diese umfasst unter anderem:

- Lagerplatz
- QR Code (Artikelnummer)
- Artikelname

Durch die Auslagerung der Daten in eine externe Struktur wird eine deutlich höhere Flexibilität erreicht. Neue Materialien können ohne Anpassung der SPS ergänzt werden, wodurch das System leichter erweiterbar ist.

#### 3.2.2 Aufbau der Programmanwendung

Zur Verarbeitung der Materialdaten wurde eine eigene Programmanwendung entwickelt. Diese stellt das zentrale Bindeglied zwischen der Datenspeicherung und der SPS dar.

Der grundlegende Aufbau der Anwendung gliedert sich in zwei Funktionsbereiche:

- Verarbeitungsebene zur Einlesung und Aufbereitung der Materialdaten
- Auftragsebene zur Kommunikation mit der SPS

Innerhalb der Verarbeitungsebene werden die Materialdaten eingelesen und in eine geeignete Struktur überführt. Hierbei werden die relevanten Informationen extrahiert und für die weitere Nutzung vorbereitet.

Die Auftragsebene baut auf diesen aufbereiteten Daten auf und übernimmt die Kommunikation mit der **SPS**. In diesem Schritt werden die notwendigen Steuerinformationen generiert und an die Steuerung übertragen, wodurch der Auslagerprozess ausgelöst wird.

#### 3.2.3 Verarbeitung der Materialdaten

Die Verarbeitung der Materialdaten stellt einen zentralen Bestandteil der entwickelten Programmanwendung dar. Ziel ist es, die eingelesenen Daten aus der externen Datenstruktur auszuwerten und für die weitere Verwendung im System bereitzustellen.

```
def Einlesen_CSV(plc):  
    # Einlesen der bestehenden CSV Tabelle  
    table = pd.read_csv('CSV_Daten.csv', sep=";")  
    table.columns = table.columns.str.strip()
```

Abbildung 3.5: Einlesen und Vorverarbeitung der CSV-Datei mit pandas

Die Materialdaten werden zunächst aus einer externen **CSV-Datei** eingelesen. Hierfür wurde eine eigene Funktion `Einlesen_CSV` implementiert, welche die Daten mit Hilfe der Bibliothek `pandas` in eine strukturierte Tabellenform überführt. In Abbildung 3.5 ist die entsprechende Umsetzung in Python dargestellt.

Dabei wird die Bibliothek `pandas` unter der Abkürzung `pd` verwendet. Sie ermöglicht eine effiziente Verarbeitung tabellarischer Datenstrukturen. Insbesondere können Daten direkt in eine Tabellenform eingelesen werden, ohne dass explizite Schleifen erforderlich sind. Dadurch wird die Implementierung vereinfacht und gleichzeitig die Lesbarkeit des Codes verbessert.

Im nächsten Schritt erfolgt ein Abgleich der eingelesenen Daten mit den aktuellen Zuständen der Anlage. Dazu werden relevante Informationen, wie beispielsweise Lagerplatzkoordinaten sowie die Artikelnummer, aus der **SPS** ausgelesen. Diese Werte dienen als Grundlage für die Aktualisierung der Materialdaten.

Anhand der Artikelnummer wird innerhalb der geladenen Tabelle ein entsprechender Eintrag identifiziert. Dieser Vorgang entspricht einem sogenannten **Matching** bzw. einer Zuordnung von Datensätzen. Wird ein passender Eintrag gefunden, erfolgt eine Aktualisierung der zugehörigen Lagerplatzinformationen innerhalb der CSV-Struktur.

```
# Überprüfung ob Code vorhanden (RFID vgl QR Code)
code_maske = table["QR Code"].astype(str).str.strip() == str(Artikelnummer).strip()
row = table.loc[code_maske]
```

Abbildung 3.6: Filterung der Materialdaten anhand der Artikelnummer

In Abbildung 3.6 ist der Abgleich der eingelesenen Daten mit den von der SPS bereitgestellten Werten dargestellt. Hierbei werden die Artikelnummern aus beiden Systemen miteinander verglichen.

Da die Daten aus unterschiedlichen Quellen stammen, wird zunächst eine Umwandlung in einen einheitlichen Datentyp durchgeführt. Dies erfolgt durch einen sogenannten *String-Cast*, wodurch sichergestellt wird, dass die Vergleichsoperation korrekt durchgeführt werden kann.

Das Ergebnis dieses Vergleichs wird in einer logischen Maske (z. B. `code_maske`) gespeichert. Diese Maske dient anschließend dazu, gezielt die entsprechende Zeile innerhalb der Tabelle auszuwählen.

Nach erfolgreicher Verarbeitung werden die aktualisierten Daten wieder in die CSV-Datei zurückgeschrieben. Dadurch bleibt die Datenbasis stets aktuell und reflektiert den Zustand des realen Lagersystems.

```
# Joining anhand des Codes
if not row.empty:
    table.loc[code_maske, "Lagerplatz x"] = Lagerplatz_x
    table.loc[code_maske, "Lagerplatz y"] = Lagerplatz_y
# Speichern der neuen Tabelle mit aktualisierten Werten
table.to_csv("CSV_Daten.csv", sep=";", index=False)
```

Abbildung 3.7: Aktualisierung der Lagerplatzdaten in der CSV-Datei

Mithilfe der `.loc`-Funktion wird gezielt auf die zuvor gefilterten Einträge innerhalb der Tabelle zugegriffen. Dies ermöglicht eine direkte Aktualisierung der Lagerplatzdaten für das jeweilige Material.

Die neuen Werte, beispielsweise `Lagerplatz_x` und `Lagerplatz_y`, werden aus der SPS übernommen und in die entsprechenden Spalten der CSV-Datei eingetragen.

Nach der erfolgten Aktualisierung wird die Tabelle wieder in die CSV-Datei zurückgeschrieben. Dadurch bleibt die Datenbasis konsistent und spiegelt jederzeit den aktuellen Zustand des Lagersystems wider.

#### 3.2.4 Auslagerprozess innerhalb der Programmanwendung

Für die Umsetzung des auftragsbasierten Auslagerprozesses wurde die Funktion `Auslagern` entwickelt. Diese übernimmt die Aufgabe, einen durch den Scanner erfassten Code auszuwerten und die entsprechenden Steuerinformationen für die Anlage zu generieren.

Zunächst wird die bestehende CSV-Datei eingelesen und anhand des übergebenen Codes gefiltert. Der Code dient dabei als eindeutige Identifikation des Materials. Wird ein entsprechender Datensatz gefunden, werden die zugehörigen Lagerplatzinformationen extrahiert<sup>3.8</sup>.

```
row = table.loc[table["QR Code"].astype(str) == Code_Scanner]
```

Abbildung 3.8: Zuordnung eines gescannten Codes zu einem Materialdatensatz

Im Anschluss erfolgt eine Überprüfung, ob für das ausgewählte Material gültige Lagerplatzdaten vorliegen. Ist dies nicht der Fall, wird der Prozess abgebrochen, um fehlerhafte Zustände zu vermeiden.

Sind gültige Daten vorhanden, werden diese für die weitere Nutzung aufbereitet und an die **SPS** übergeben. Hierfür wird eine separate Funktion verwendet, welche die Daten in das entsprechende Format überführt und in die relevanten Speicherbereiche der Steuerung schreibt<sup>3.9</sup>.

```
# Schreiben der Werte in SPS Code über Snap7  
Schreiben(plc, Lagerplatz_x, Lagerplatz_y, Material)
```

Abbildung 3.9: Übergabe der berechneten Daten an die SPS



Zusätzlich wird jede Durchführung eines Ein- oder Auslagerprozesses protokolliert. Diese Historie ermöglicht eine Nachvollziehbarkeit der durchgeführten Aktionen und dient der Analyse des Systemverhaltens.

#### 3.2.5 Einlesen von Daten aus der SPS

Zur Kommunikation mit der Steuerung wurde eine Funktion `Einlesen` implementiert. Diese greift über die Snap7-Schnittstelle auf definierte Datenbausteine der **SPS** zu und liest die dort abgelegten Werte aus.

Dabei werden insbesondere Koordinatenwerte sowie Artikelinformationen in Python-Variablen überführt und für weitere Verarbeitungsschritte bereitgestellt.

#### 3.2.6 Übertragung von Daten an die SPS

Für die Übergabe von Steuerinformationen an die Anlage wurde die Funktion `Schreiben` realisiert. Diese transformiert die in Python vorliegenden Daten in das benötigte Byteformat und überträgt diese in die entsprechenden Datenbereiche der **SPS**.

Durch das Setzen dieser Werte wird der eigentliche Auslagerprozess innerhalb der Steuerung ausgelöst.

#### 3.2.7 Erfassung von Scannerdaten

Die Erfassung der Auftragsdaten erfolgt über einen seriell angebundenen Scanner. Hierfür wurde eine Funktion zur kontinuierlichen Datenerfassung implementiert. Die empfangenen Daten werden als Zeichenkette eingelesen und anschließend bereinigt.

```
def Scanner():  
    while True:  
        data = ser.readline().decode('latin-1', errors='ignore').strip()  
        data = ''.join(c for c in data if c.isdigit())
```

Abbildung 3.10: Einlesen und Bereinigung der Scannerdaten

In Abbildung 3.10 ist der Prozess des Einlesens und der anschließenden Verarbeitung der Scannerdaten dargestellt. Die Daten werden über eine serielle Schnittstelle empfangen und zunächst als Rohdaten eingelesen.

Da die übertragenen Daten neben der eigentlichen Information zusätzliche Zeichen enthalten können, erfolgt im nächsten Schritt eine Bereinigung der Daten.

Hierbei werden ausschließlich numerische Zeichen extrahiert, sodass ein eindeutiger Code zur weiteren Verarbeitung entsteht.

Durch diese Verarbeitung wird sichergestellt, dass nur gültige und verwertbare Daten im System verwendet werden. Der bereinigte Code dient anschließend als Grundlage für die Identifikation des entsprechenden Materials innerhalb der Datenbank und wird weiter im Auslagerprozess verwendet.

Die kontinuierliche Überwachung der Schnittstelle stellt sicher, dass eingehende Daten unmittelbar verarbeitet werden können. Eine zukünftige Erweiterung des Systems sieht hierbei eine Parallelisierung der Datenverarbeitung vor.

Im folgenden Kapitel werden die notwendigen Anpassungen im SPS Programm erläutert um eine erfolgreiche Auftragssteuerung zu gewährleisten.

## 3.3 Anpassungen in der SPS

Nach der Implementierung der Auftragssteuerung musste sichergestellt werden, dass der Auslagerprozess mit dem entwickelten Python-Softwarekonzept synchronisiert wird. Um diese Integration zu realisieren, wurden sowohl software- als auch hardwareseitige Anpassungen innerhalb der SPS vorgenommen.

### 3.3.1 Datenbausteine im Kontext der auftragsbasierten Auslagerung

Das vorliegende System umfasst bereits grundlegende Prozessschrittketten für die Funktionen Einlagern, Auslagern und Umlagern. Eine detaillierte Beschreibung dieser Abläufe ist in der Dokumentation der Studienarbeit T3\_3100 enthalten.

Für die Kommunikation zwischen der externen Programmanwendung und der **SPS** wurde ein zusätzlicher Datenbaustein implementiert. Dieser ermöglicht einen strukturierten Datenaustausch über die Snap7-Schnittstelle.

Grundsätzlich erfolgt der Zugriff über Snap7 durch direkte Adressierung von Variablen anhand der Datenbaustein-Nummer sowie der zugehörigen Speicheradressen.

```
data_x_ue = plc.db_read(1, 2, 2) # int = 2 Byte  
lagerplatz_x_ue = get_int(data_x_ue, 0)
```

Abbildung 3.11: Einlesen einer Variablen aus der SPS mittels Snap7

In Abbildung 3.11 ist ein beispielhafter Einlesevorgang dargestellt. Die Funktion `db_read` ermöglicht den Zugriff auf eine Variable innerhalb eines Datenbausteins, in diesem Fall auf `Lagerplatz_x_ue`.

Die übergebenen Parameter definieren dabei die Datenbaustein-Nummer, die Startadresse im Speicher sowie die Größe des auszulesenden Datentyps. Für einen **Integer** werden standardmäßig 2 Byte verwendet. Für andere Datentypen muss die entsprechende Größe angepasst werden.

Nach diesem Prinzip werden alle erforderlichen Informationen, wie beispielsweise Lagerplatz- oder Artikelnummern, aus den bestehenden Datenbausteinen ausgelesen und in der Programmanwendung weiterverarbeitet.

Für die Realisierung eines zuverlässigen **Handshakes** zwischen **SPS** und Python-Anwendung wurden zusätzliche Signale eingeführt. Hierzu wurde ein weiterer Datenbaustein angelegt, der speziell der Kommunikation dient.

Um einen direkten Zugriff auf die Speicheradressen zu ermöglichen, wurde der **optimierte Bausteinzugriff** deaktiviert. Nach dieser Anpassung musste der Datenbaustein neu übersetzt werden, um eine korrekte Adressierung zu gewährleisten.



The screenshot shows a table titled 'DB\_Auftragsmanagement' with four columns: an index, a 'Name' column, a 'Datentyp' column, and an 'Offset' column. The table contains nine rows of data. The first row is a header for a 'Static' section. The subsequent rows define variables: 'i\_soll\_Position\_x' (Int, 0.0), 'i\_soll\_Position\_y' (Int, 2.0), 'b\_Event\_Aufr' (Bool, 4.0), 'b\_V\_Bestätigung\_btn' (Bool, 4.1), 's\_MaterialName' (String[20], 6.0), 'b\_EinlagerungDone' (Bool, 28.0), 'b\_UmlagerungDone' (Bool, 28.1), and 'b\_Python\_Bestätigung' (Bool, 28.2). Each row has a small icon to its left, and the 'i\_soll\_Position\_y' row has a list icon to its right.

|   | Name                 | Datentyp   | Offset |
|---|----------------------|------------|--------|
| 1 | Static               |            |        |
| 2 | i_soll_Position_x    | Int        | 0.0    |
| 3 | i_soll_Position_y    | Int        | 2.0    |
| 4 | b_Event_Aufr         | Bool       | 4.0    |
| 5 | b_V_Bestätigung_btn  | Bool       | 4.1    |
| 6 | s_MaterialName       | String[20] | 6.0    |
| 7 | b_EinlagerungDone    | Bool       | 28.0   |
| 8 | b_UmlagerungDone     | Bool       | 28.1   |
| 9 | b_Python_Bestätigung | Bool       | 28.2   |

Abbildung 3.12: Datenbaustein für das Auftragsmanagement

In Abbildung 3.12 sind die neu eingeführten Variablen mit ihren jeweiligen Adressen dargestellt.

Die Variablen `i_Soll_Position` sowie `s_MaterialName` werden durch die Python-Anwendung gesetzt und enthalten die relevanten Informationen für den Auslagerprozess, wie beispielsweise Artikelnummer und Lagerplatzkoordinaten.

Zusätzlich wurden Steuer- und Statussignale wie `b_EinlagerungDone`, `b_UmlagerungDone` sowie `b_Python_Bestaetigung` eingeführt. Diese dienen der Umsetzung eines zuverlässigen Handshakes und gewährleisten, dass die Daten nach jedem Ein- oder Umlagerprozess korrekt verarbeitet und synchronisiert werden.

Die Variable `b_Event_Auftr` ermöglicht der SPS das Erkennen eines neuen Auftrags. Über die Variable `b_V_Bestaetigung_btn` kann ein Bestätigungssignal vom **HMI** verarbeitet werden, welches den Start des Auslagerprozesses auslöst.

Nach der Implementierung des Datenbausteins konnte im nächsten Schritt die Anpassung der Prozessschrittkette erfolgen.

#### 3.3.2 Anpassung der Prozessschrittkette “Auslagern”

Im vorherigen Abschnitt wurden die neu eingeführten Variablen bereits erläutert. Im Folgenden wird die Anpassung der Prozessschrittkette “Auslagern” innerhalb der SPS beschrieben.

Im ursprünglichen System wurde der Auslagerprozess manuell über einen Button im **HMI** gestartet. Nach der Auslösung erfolgt innerhalb der SPS ein Soll-Ist-Vergleich der Ziel- und Ist-Positionen in X- und Y-Richtung. Anschließend wird das entsprechende Material aus dem Lager entnommen.

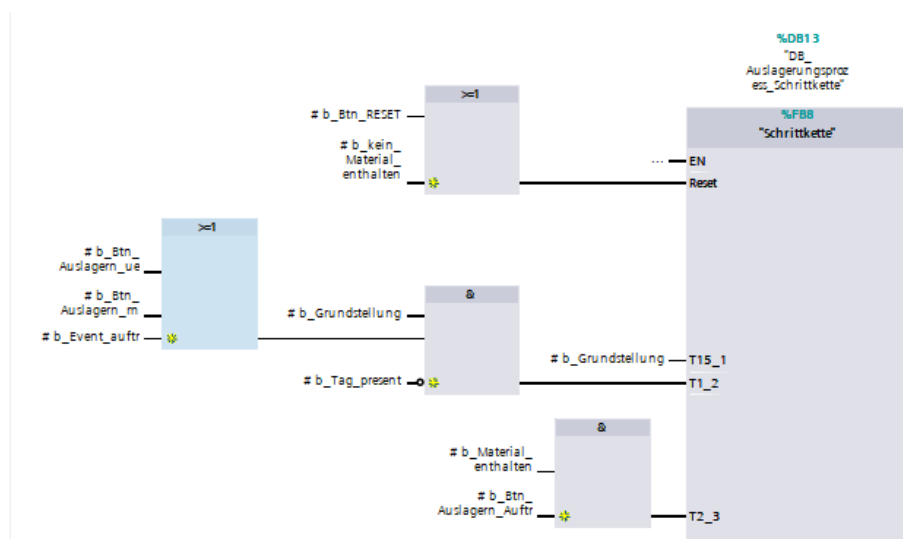


Abbildung 3.13: Prozessablauf der Auslagerung in der SPS

Im Zuge der Erweiterung hin zu einer auftragsbasierten Steuerung wurde dieser Ablauf angepasst und um zusätzliche Prozessschritte ergänzt3.13.

Zunächst wurde ein Triggermechanismus implementiert, bei dem die **SPS** den Scanner gezielt aktiviert.

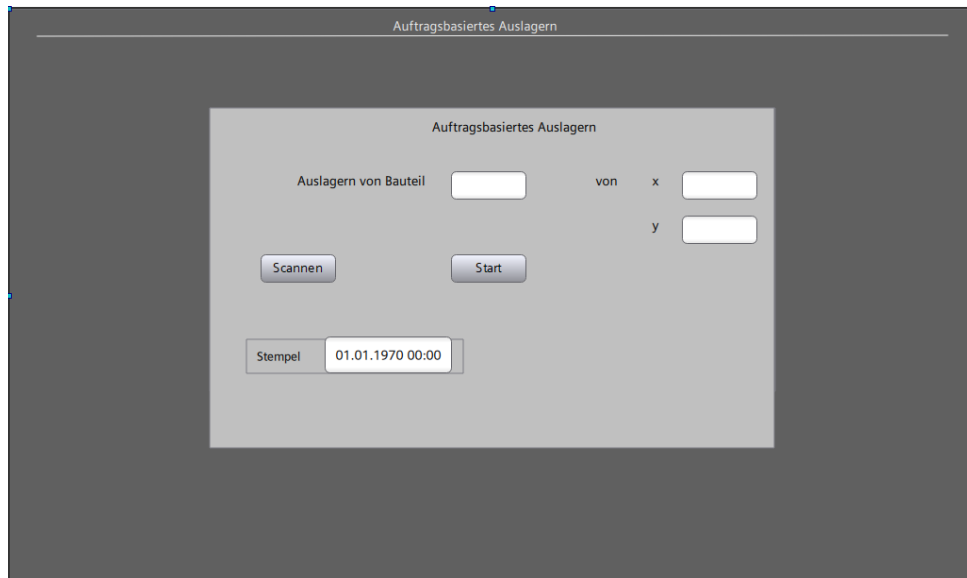


Abbildung 3.14: Darstellung des auftragsbasierten Auslagerungsprozesses im HMI

Hierzu wird ein entsprechendes Signal gesetzt, wodurch der Scanner einen Scanvorgang ausführt und den erfassten Code an die Programmanwendung übergibt (vgl. Abbildung 3.14).

Auf Basis dieses Codes erfolgt die Verarbeitung innerhalb der Programmanwendung. Die daraus resultierenden Steuerinformationen werden anschließend zurück an die SPS übertragen.

Zur eigentlichen Auslösung des Auslagerprozesses wurde die Variable `b_Event_Auftr` eingeführt. Diese wird durch die externe Programmanwendung gesetzt und signalisiert der SPS das Vorliegen eines neuen Auftrags.

Nach Erkennung dieses Signals wird der Prozess vorbereitet, jedoch noch nicht unmittelbar ausgeführt. Eine zusätzliche manuelle Freigabe durch den Bediener bleibt erforderlich. Diese erfolgt über das **HMI** mittels der Variable `b_V_Bestaetigung_btn` (vgl. Abbildung 3.14). Erst nach dieser Bestätigung wird der Auslagerprozess tatsächlich gestartet.

Durch diese Erweiterung konnte die bestehende Prozessschrittkette um eine auftragsbasierte Steuerung ergänzt werden. Gleichzeitig bleibt die Kontrolle über den Ablauf durch die manuelle Freigabe erhalten, wodurch die Betriebssicherheit der Anlage gewährleistet wird.

## Literatur

- [1] Datalogic AG. *Reference Guide - Factory Automation*. Zugriff am 07. Juni 2026. 2014. URL: [https://www.datalogic.com/upload/Flipbook/Reference%20guide%20ID\\_Factory\\_Automation/files/assets/common/downloads/publication.pdf](https://www.datalogic.com/upload/Flipbook/Reference%20guide%20ID_Factory_Automation/files/assets/common/downloads/publication.pdf).
- [2] Forum, USB Implementers. *Universal Serial Bus Specification*. Zugriff am: 26.05.2026. 2023. URL: <https://www.usb.org>.
- [3] Langmann, R. *Vernetzte Systeme für die Automatisierung 4.0*. 1. Auflage. Carl Hanser Verlag München, 2021. ISBN: 978-3-446-46984-6.
- [4] Li, H. und Bao, Z. *Transport Layer for Computer Networks*. Springer, 2026. ISBN: 978-981-95-7374-5.
- [5] Mikrocontroller.net. *RS-232*. Zugriff am: 26.05.2026. 2024. URL: <https://www.mikrocontroller.net/articles/RS-232>.
- [6] Molenaar, G. und Preeker, S. *snap7 - Python bindings for the snap7 library*. Zugriff am 08. Juni 2026. 2013-2026. URL: <https://python-snap7.readthedocs.io/en/3.0.0/>.
- [7] Muchna, C. u. a. *Grundlagen der Logistik*. Springer Gabler Wiesbaden, 2026. ISBN: 978-3-658-52420-3.
- [8] Seitz, M. *Speicherprogrammierbare Steuerungen in der Industrie 4.0*. 5. Auflage. Carl Hanser Verlag München, 2021. ISBN: 978-3-446-47002-6.
- [9] Studieninstitut, Deutsches eLearning. *E-V-A-Prinzip*. Zugriff am 04. November 2025. 2025. URL: <https://www.delst.de/de/lexikon/e-v-a-prinzip/>.
- [10] Uniqode Phygital. *QR-Code vs. Barcode - was ist besser für Ihr Unternehmen?* Zugriff am 07. Juni 2026. 2026. URL: <https://www.the-qr-code-generator.com/de/blog/barcode-vs-qr-code>.
- [11] Weyrich, M. *Industrielle Automatisierungs- und Informationstechnik*. 1. Auflage. Springer-Verlag GmbH Deutschland, 2023. ISBN: 978-3-662-56355-7.